

— Objetivos —

Os alunos deverão desenvolver as seguintes competências e habilidades:

- Compreender o conceito de alocação dinâmica de memória
- Implementar e manipular estruturas de dados utilizando alocação dinâmica em blocos

— Exercícios —

Exercícios sobre Alocação Dinâmica:

1) Fazer uma função que gere e retorne uma string *S* com *N* caracteres *C*. O protótipo da função é dado por:

*char * gerastr(int n, char c)*

2) Fazer uma função que gere e retorne uma string com *N* caracteres aleatórios. Cada caracter pode ser aleatoriamente escolhido entre qualquer letra maiúscula, minúscula ou dígito. O protótipo da função é dado por:

*char * aleatoria(int n)*

3) Implemente a função

*char * ultimo_nome(char * nome_completo)*

que recebe como parâmetro uma string com o nome completo de uma pessoa e retorna uma string alocada dinamicamente com o último nome encontrado. A string alocada deverá ter o tamanho exato para comportar os caracteres do último nome, além do caractere nulo ('\0').

Faça um programa que leia 5 strings com os nomes e mostre os últimos nomes, um por linha. Por exemplo, para as entradas

```
Carlos Alberto Silva
Sebastião Lazaroni
Carlos Alberto Sant'ana
Mário Jorge Lobo Zagallo
Vanderlei Luxemburgo
```

o programa deve produzir a seguinte saída

```
Silva
Lazaroni
Sant'ana
```

4) Faça uma função que receba por parâmetro um valor inteiro n e retorne um vetor de n inteiros alocado dinamicamente com os n primeiros termos da série de Fibonacci. Lembre-se que os 2 primeiros elementos da série de Fibonacci são 0 e 1 e cada próximo termo é a soma dos 2 anteriores a ele. Por exemplo, os 11 primeiros termos da série de Fibonacci são:

0 1 1 2 3 5 8 13 21 34 55 ...

O protótipo da função é dado por:

*int * fibonacci(int n)*

5) Faça uma função que receba por parâmetro um valor inteiro n e retorne um vetor de n inteiros alocado dinamicamente com os n primeiros números primos. Faça a função, inicializando as duas primeiras posições com 2 e 3, a verificação da primalidade dos números posteriores seja feita dividindo cada número apenas pelos primos menores que ele, que já estarão no vetor. Se o número não tiver divisores, ocupará a próxima posição livre do vetor. O protótipo da função é dado por:

*int * primos(int n)*

5) Crie uma função que receba como parâmetros dois vetores de inteiros: $x1$ e $x2$ e as suas respectivas quantidades de elementos: $n1$ e $n2$. A função deverá retornar um ponteiro para um terceiro vetor, $x3$, alocado dinamicamente, contendo a intersecção de $x1$ e $x2$ e usar o ponteiro qtd para retornar o tamanho de $x3$. Assuma que em um mesmo vetor não haverá elementos repetidos. Sendo $x1=\{1,3,5,6,7\}$ e $x2=\{1,3,4,6,8\}$, $x3$ irá conter $\{1,3,6\}$. O protótipo da função é dado por:

int interseccao(int *x1, int *x2, int n1, int n2, int* qtd)*

7) Crie uma função que receba como parâmetros dois vetores de inteiros: $x1$ e $x2$ e as suas respectivas quantidades de elementos: $n1$ e $n2$. A função deverá retornar um ponteiro para um terceiro vetor, $x3$, alocado dinamicamente, contendo a união de $x1$ e $x2$ e usar o ponteiro qtd para retornar o tamanho de $x3$. Assuma que em um mesmo vetor não haverá valores repetidos, mas um valor contido em $x1$ poderá também estar contido em $x2$. Nesse caso, apenas uma ocorrência deve

ser colocada em x2. Sendo x1={1,3,5,6,7} e x2={1,3,4,6,8}, x3 irá conter {1,3,5,6,7,4,8}. O protótipo da função é dado por:

int uniao(int *x1, int *x2, int n1, int n2, int* qtd)*

8) Considere a declaração da struct a seguir, que contém o valor, linha e coluna de um elemento não nulo de uma matriz.

```
struct naonulo{
    int valor;
    int linha;
    int coluna;
};
typedef struct naonulo NAONULO;
```

Faça uma função naonulos que recebe uma matriz de inteiros M[5][5] e aloca um vetor de naonulo de tamanho igual à quantidade de elementos diferentes de 0 na matriz, e coloca nele os elementos não nulos da matriz recebida, retornando, como valor de retorno da função, o endereço para o vetor alocado. A função deve retornar NULL se não há houverem elementos não nulos. Além disso, a função deverá retornar o número de elementos não nulos através de um argumento. O protótipo da função é dado por:

NAONULOS valores_nao_nulos(int M[5][5], int *num_nao_nulos)*

9) Considere o cadastro de clientes de uma empresa, onde cada cliente é representado pelo seguinte tipo:

```
struct cliente {
    int codigo;
    char nome[50];
    DATA nasc;
};
typedef struct cliente CLIENTE;
```

O campo nascimento, que representa o dia, mês e ano de nascimento do cliente, é um campo do tipo DATA, descrito a seguir:

```
struct data {
    int dia;
    int mes;
    int ano;
```

```
};  
typedef struct data DATA;
```

Faça uma função que receba como parâmetros um vetor com um conjunto de dados do tipo CLIENTE (e o seu tamanho) e data atual. A função deverá:

- Calcular o número de clientes maiores de idade e retorne este valor através de um argumento;
- Retornar um novo vetor de estruturas CLIENTE (alocado dinamicamente e com o tamanho exato) que contém todos os clientes maiores de idade;

A função deve retornar NULL se não há houverem clientes maiores de idade. O protótipo da função é dado por:

CLIENTE maior_de_idade(CLIENTE *vet, int num_clientes, DATA atual, int *num_maiores_idade)*

10) Considere um tipo que representa as coordenadas, X e Y, de pontos no plano, definido pela estrutura:

```
struct ponto {  
    float x;  
    float y;  
};  
typedef struct ponto PONTO;
```

Faça um programa que leia n elementos do tipo PONTO e determine quais desses pontos se encontram dentro de uma determinada região retangular. O programa deverá ler um valor n e criar um dinamicamente um vetor de n elementos do tipo PONTO. Após o programa deverá ler as coordenadas dos n pontos e de dois pontos (x1, y1) e (x2, y2).

Após, faça uma função, conforme o seguinte protótipo

*PONTO *filtra_pontos (PONTO *p, int npontos, float x1, float y1, float x2, float y2, int *qtd);*

que receba o vetor P e as coordenadas dos dois pontos e retorne um vetor alocado dinamicamente com os pontos de P que se encontram dentro da determinada região retangular. O vetor retornado deverá conter apenas os pontos cujas as coordenadas x e y obedeçam a regra: $x_1 \leq x < x_2$ e $y_1 \leq y < y_2$.

Por exemplo, supondo que o vetor de entrada da função tenha o conteúdo:

```
1.2 3.2  
2.4 1.5  
3.1 2.3  
9.1 5.5
```

2.1 4.9

7.5 4.8

e os valores dos parâmetros x1, y1, x2 e y2 sejam respectivamente 3.0, 2.0, 9.5, e 5.0, o vetor de saída deverá conter o seguintes pontos:

3.1 2.3

7.5 4.8

11) Considere um vetor de structs do tipo ALUNO que contenha: a turma, a matrícula e a nota de de um aluno:

INF1001	9911234-9	7.3
INF1001	9821234-9	8.7
CRE0700	9926789-1	10.0
CRE0700	9010087-2	9.2
INF1620	8820324-3	3.4
INF1620	9210478-5	7.2
INF1620	9020256-8	6.7

Faça uma função em C, conforme o protótipo

*ALUNO *media (ALUNO *a, int n alunos, char codigo[15], float *media, int *qtd);*

que receba um vetor do tipo ALUNO (o tamanho deve também ser passado como parâmetro) e o código da disciplina e retorne a média das notas obtidas pelos alunos nessa disciplina. A função deverá retornar ainda um vetor alocado dinamicamente com todos os alunos dessa disciplina. Se no vetor não existir alunos da disciplina dada, a função deverá retornar NULL e uma média igual a -1.

Faça um programa que leia um valor N aloque um vetor para N structs do tipo ALUNO, faça a leitura dos dados e requisite um código de disciplina. O programa deve chamar a função media() e apresentar a média das notas da respectiva disciplina (ou -1 se a mesma não existir aluno algum). Por exemplo, para a struct dada como exemplo no início desta questão, caso o usuário digitar a disciplina de código CRE0700, a média que deve ser mostrada na tela é 9,6 ((10 + 9,2)/2). No caso de digitar um código de disciplina inexistente, o programa deve mostrar na tela o valor -1.

12) O cadastro dos motoristas de uma empresa de transportes utiliza um vetor de estruturas com o nome (char nome[51]), número de matrícula (inteiro) e o número de multas (inteiro) de seus motoristas. Faça uma função, com o protótipo

*MOTORISTAS *piores_motoristas(MOTORISTAS m[], int num_motoristas, int N);*

que receba o vetor de motoristas, o número total de motoristas da empresa e um número N (onde $N < \text{num_motoristas}$) e retorne um vetor alocado dinamicamente com os N piores motoristas da empresa (com maior número de multas). Os motoristas devem estar organizados em ordem decrescente em relação ao número de multas e se dois motoristas tiverem o mesmo número de multas eles deverão ser organizados em ordem alfabética.

Faça também o programa principal que deverá ler um valor M e alocar um vetor de M motoristas, faça a leitura dos dados dos M MOTORISTAS. Após o programa deverá ler o valor de N e efetuar a chamada da função, e deverá escrever o vetor com os piores motoristas. Por exemplo, para os seguintes dados dos motoristas (M=6)

Marcos Gomes Silva	2331	8
Rodrigo Correia Ribeiro	145	4
Alice Oliveira Santos	1465	6
Nicolas Silva Cavalcanti	2934	7
Daniel Silva Melo	2127	9
Diogo Sousa Almeida	1814	11

e um N com valor igual a 4, o programa deve produzir a seguinte saída

1814	Diogo Sousa Almeida	11
2127	Daniel Silva Melo	9
2331	Marcos Gomes Silva	8
2934	Nicolas Silva Cavalcanti	7

Solução dos Exercícios sobre Alocação Dinâmica

Alocação Dinâmica - Trabalho Discente

Efetivo - TDE

1) Faça uma função que receba uma string com valores inteiros separados por vírgulas e retorne um vetor com esses valores. A função implementada deverá obrigatoriamente seguir o protótipo que é definido abaixo.

`int * separa(char str[])`

Exemplo:

String	Vetor				
"10,15,245,11,5,1243"	10	15	245	11	

Faça ainda o programa principal que leia a string e chame a função implementada. Após, esse programa deverá escrever o vetor com todos os valores retornados.

OBSERVAÇÃO: não é permitida a utilização de funções de manipulação de strings.

2) Implemente uma função em C que receba um vetor de estruturas contendo informações sobre transações bancárias. Cada estrutura possui os seguintes campos:

```
typedef struct data {
    int dia;
    int mes;
    int ano;
}DATA;

typedef struct transacao {
    Data data;
    float valor;
    char descricao[100];
}TRANSACAO;
```

Desenvolva uma função chamada `filtrarPorData` que receba como parâmetros:

- Um vetor de estruturas struct `TRANSACAO` contendo todas as transações bancárias.
- O tamanho do vetor.
- Uma data inicial e uma data final, representadas por estruturas do tipo `DATA`

A função `filtrarPorData` deverá retornar:

- Um novo vetor contendo as transações que estão dentro do intervalo de datas especificado.
- Modificar um inteiro com o número de transações encontradas dentro do intervalo.

A função obrigatoriamente deverá seguir o seguinte protótipo:

```
TRANSACAO * filtrarPorData(TRANSACAO *transacoes, int n, DATA inicio, DATA fim, int *nt);
```

Implemente o programa principal em C que chame a função `filtrarPorData` para filtrar as transações entre duas datas específicas e imprime as transações retornadas pela função.

Solução do TDE - Alocação Dinâmica

— Leitura Complementar —

- Minha Biblioteca
 - DAMAS, Luís. Linguagem C
 - Capítulo 8 e 12
 - EDELWEISS, Nina. Algoritmos e programação com exemplos em Pascal e C
 - Capítulo 14
 - MANZANO, José Augusto N. G. Linguagem C acompanhada de uma xícara de café
 - Capítulo 5
 - MANZANO, José Augusto N. G. Programação de computadores com C/C++
 - Seções 5.2 e 5.6
 - PINHEIRO, Francisco de Assis Cartaxo. Elementos de programação em C
 - Seções 10.1 a 10.3
 - SOFFNER, Renato. Algoritmos e programação em linguagem C
 - Capítulo 8
- BV
 - DEITEL, Paul J.; DEITEL, Harvey M.; CAETANO, César Augusto Cardoso. C : como programar
 - Capítulo 7 e Seção 12.3
 - MIZRAHI, Victorine Viviane. Treinamento em linguagem C
 - Capítulo 9